



Senior Project Report

Vbot

Peerapat Thubthimthong,
Natthaphat Kraiwong,
Tanai Luchino Forges Davanzati

Dr. Kwankamol Nongpong (Advisor)

Senior Project Approval

Project title: Vbot
Academic Year: 2/2024
Authors: Peerapat Thubthimthong (6510190)
Natthaphat Kraiwong (6510249)
Tanai Luchino Forges Davanzati (6510210)
Project Advisor: Dr Kwankamol Nongpong

The Senior Project committee's cooperation between the Department of Computer Science and Information Technology, Vincent Mary School of Engineering, Science and Technology, Assumption University had approved this Senior Project. The Senior Project in partial fulfilment of the requirement for the degree of Bachelor of Science in Computer Science and Information technology.

Approval Committee:

.....
(Dr Kwankamol Nongpong)
Project Advisor

.....
(Asst. Prof. Dr. Rachsuda Setthawong)
Committee Member

.....
(Asst. Prof. Dr. Darun Kesrarat)
Committee Member

Abstract

This research presents Vbot, an innovative AI-powered system designed to create interactive virtual clones of VTubers (Virtual YouTubers). The system integrates multiple cutting-edge technologies: StyleTTS2 for voice synthesis, Talking Head Anime 4 (THA4) for avatar animation, and large language models via containerized Ollama for conversational capabilities. The project addresses critical technical challenges in voice reproduction, real-time animation, and personality-consistent dialogue generation. Through extensive testing and optimization, Vbot demonstrates the feasibility of creating engaging AI-driven VTuber experiences while highlighting both the potential and current limitations of virtual character simulation technology. Performance analysis reveals consistent frame rates of 15 FPS on recommended hardware and response times under one second, making real-time interaction viable. While the system successfully creates recognizable VTuber clones, it also illuminates important considerations regarding data quality, hardware requirements, and ethical implications in AI-driven entertainment.

Table Of Contents

Senior Project Approval	2
Abstract	2
Table Of Contents	4
Table of Figures	6
Chapter 1: Introduction	7
1.1 Background	7
1.2 Problem Statement	7
1.3 Scope of the project	8
Chapter 2: Related Work	8
2.1 Virtual Character Animation	8
2.2 Text-to-Speech Systems	9
2.3 Conversational AI	10
2.4 Commercial AI Character Platforms (CharacterAI)	10
Chapter 3: Proposed Methodology	11
3.1 Model Selection	11
3.2 Data Collection	13
3.3 Data Processing	14
3.4 Training Approach	15
3.4.1 TTS Model Fine-tuning	15
3.4.2 Avatar Implementation	16
3.4.2.1 THA4-Based Approach	16
3.4.2.2 Live2D Approach	17
3.4.2.3 Approach Comparison	18
3.4.3 Avatar Integration	18
Chapter 4: Design of The System	20
4.1 Speech-To-Text (STT)	20
4.2 Large-Language-Model (LLM)	21
	4

4.3 Emotion Classification	24
4.4 Text-To-Speech (TTS)	25
4.5 Avatar Animation (THA4)	27
4.5.1 System Architecture	28
4.5.2 Integration Implementation	29
4.5.3 Animation Implementation	30
4.5.4 Technical Implementation Details	31
4.6 User Interface	32
4.6.1 Initial Prototype Implementation	32
4.6.2 THA4 Integration and Voice Synthesis	33
4.6.3 Final Implementation	34
Chapter 5: Result	35
5.1 Delivery outcome	35
5.2 System Performance	39
5.3 Detailed Performance Analysis	46
Chapter 6: Conclusion	48
References	50
Appendix A: User Manual for Vbot	51
A.1 System Requirements	51
A.2 Installation Guide	51
A.3 Using the Application	53
A.4 Troubleshooting	54

Table of Figures

Figure 1 Workflow	20
Figure 2 comparison THA4 and Live2d	29
Figure 3 Initial Prototype Implementation	32
Figure 4 THA4 Integration and Voice Synthesis	33
Figure 5 Final Implementation	34
Figure 6 Result chatbot	35
Figure 7 The delivery outcome : Main UI	35
Figure 8 The delivery outcome : Input UI and Buttons	36
Figure 9 The delivery outcome : Avatar (Amelia Watson)	36
Figure 10 Outcome emotion	37
Figure 11 The delivery outcome : Chat History	37
Figure 12 The delivery outcome : Response Subtitles	38
Figure 13 The delivery outcome : Avatar (Ike Eveland)	38
Figure 14 The delivery outcome : Chat History(Ike Eveland)	39
Figure 15 Overall Summary Evaluation	42
Figure 16 Emotion Recognition Results	44
Figure 17 Average Confidence When Correctly Identified	44
Figure 18 Recognition Accuracy	45
Figure 19 Avatar face : Amelia Watson	45
Figure 20 Avatar face : Ike Eveland	46

Chapter 1: Introduction

1.1 Background

Virtual YouTubers (VTubers) have emerged as a significant cultural phenomenon in online content creation, particularly in Japan and increasingly worldwide. These digital avatars, controlled by human performers, have amassed substantial followings and created deep parasocial relationships with their audiences. Fans often develop strong emotional connections to VTubers, appreciating their personalities, voices, and appearances.

The growing popularity of VTubers has created a unique opportunity for technology to bridge the gap between content creators and their audiences. While fans can watch VTuber content, they rarely get to experience direct, personalized interactions with their favorite virtual personalities. This limitation in the fan experience presents an opportunity for AI technology to create more immersive and interactive experiences.

Recent advancements in machine learning have made it possible to create increasingly realistic simulations of human speech, facial expressions, and conversational abilities. These technologies can be applied to create virtual clones of VTubers that can interact with fans directly, extending the VTuber experience beyond passive consumption of pre-recorded content.

1.2 Problem Statement

Fans of VTubers currently have limited ways to interact with their favorite virtual personalities. Most interaction is limited to watching streams, commenting on videos, or participating in structured fan events. There is no way for fans to have personalized, one-on-one conversations with VTubers that feel authentic and reflect the personality, voice, and visual style of the original VTuber.

Creating a convincing VTuber clone presents several technical challenges:

- Accurately reproducing the VTuber's voice with proper emotional inflection and speaking style
- Animating a virtual avatar that matches the appearance and movement style of the original VTuber

- Developing conversational AI that can mimic the VTuber's personality and speaking patterns .Ensuring real-time performance that allows for natural conversation flow
- Collecting and processing high-quality training data while filtering out unusable content.

The Vbot project aims to address these challenges by creating an end-to-end system for cloning VTubers that fans can interact with through text or voice input.

1.3 Scope of the project

The Vbot project focuses on developing a comprehensive system for cloning VTubers with the following core components:

Voice Synthesis: Fine-tuning StyleTTS2 models on VTuber speech data to reproduce their unique vocal characteristics.

Avatar Animation: Implementing the Talking Head Anime 4 (THA4) system to create realistic animations of VTuber avatars based on speech and conversational context.

Conversational AI: Integrating large language models to simulate conversations in the style and personality of target VTubers.

User Interface: Developing an intuitive interface that allows fans to interact with VTuber clones through text and voice.

Data Processing Pipeline: Creating efficient workflows for collecting, filtering, and processing VTuber speech data to train high-quality voice models.

The project aims to provide a complete, end-to-end solution that can be applied to create clones of various VTubers, with a focus on high-quality voice synthesis and visual representation that captures the essence of the original virtual personalities.

Chapter 2: Related Work

2.1 Virtual Character Animation

Virtual character animation, particularly for anime-style characters, has evolved significantly in recent years. Traditional techniques required extensive manual animation, making real-time interaction impractical. Recent advances in deep learning have enabled more automated approaches.

The Talking Head Anime from a Single Image (THA)

project by Pramook Khungurn represents a significant advancement in this field. The project has progressed through several iterations, with the fourth version (THA4) used in the Vbot system. THA4 offers several key innovations:

Single Image Animation: The ability to animate a character from just one static image, eliminating the need for complex 3D models or extensive animation data.

Facial Expression Control: Precise control over facial features including eyes, eyebrows, and mouth movements to convey emotions and speech.

Body Movement: Support for upper body rotation and movement to create more natural animations.

Distillation Capability: Methods for training smaller, faster models specialized for specific character images, enabling real-time performance.

Unlike previous approaches that required extensive modeling or motion capture data, THA4 uses neural networks to learn the relationship between control parameters and desired visual outputs. This approach aligns well with the Vbot project's need for flexible character animation from limited input data.

2.2 Text-to-Speech Systems

Text-to-speech (TTS) technology has seen remarkable improvements in naturalness and expressivity with the advent of deep learning approaches. For the Vbot project, accurately reproducing a VTuber's voice with proper emotional range is crucial to creating an authentic experience.

StyleTTS2, developed by Yinghao Li et al., represents the state-of-the-art in expressive speech synthesis. Key advantages of StyleTTS2 include:

1. **Style Control:** The ability to manipulate speech prosody, emotion, and speaking style independently.

2. **Few-Shot Learning:** Capability to adapt to new voices with relatively small amounts of training data.
3. **High-Quality Output:** Generation of natural-sounding speech with appropriate rhythm and intonation.
4. **Emotion Transfer:** Preserving emotional characteristics from reference audio while generating new speech.

These capabilities make StyleTTS2 an ideal foundation for creating convincing VTuber voice clones. By fine-tuning the model on carefully curated datasets of VTuber speech, Vbot can capture the unique vocal characteristics that fans recognize and appreciate.

2.3 Conversational AI

The field of conversational AI has advanced dramatically with the development of large language models (LLMs) capable of understanding context, maintaining coherent conversations, and adapting to different communication styles. For Vbot, the conversational component needs to not only provide informative responses but also mimic the personality and speech patterns of specific VTubers.

Recent developments in this field include:

1. **Persona-Based Models:** Systems that can adopt specific personality traits and speaking styles when generating responses.
2. **Multimodal Integration:** The ability to incorporate both text and audio inputs to understand user intent more comprehensively.
3. **Real-Time Response Generation:** Techniques for generating responses quickly enough to maintain natural conversation flow.
4. **Context Management:** Methods for tracking conversation history to provide coherent, contextually appropriate responses.

The Vbot system leverages these advancements to create conversational experiences that feel authentic to the target VTuber's personality, using techniques from these areas to maintain the character's voice while providing engaging interactions.

2.4 Commercial AI Character Platforms (CharacterAI)

In recent years, several commercial platforms have emerged offering AI-powered character interactions, most notably Character.ai. This platform allows users to create and interact with AI characters based on fictional personas, historical figures, or original creations.

Character.ai represents a significant competitor in the space of AI-driven character simulation with several key features:

1. **Web-Based Interface:** Providing easy access through browsers without requiring local computation.
2. **Character Creation:** Tools for users to create their own AI characters with customizable personalities.
3. **Multi-Turn Conversations:** Support for extended dialogues with memory of previous interactions.
4. **Text-Only Interaction:** Focus on text-based communication without voice or animation components.

While Character.ai offers impressive conversational capabilities, it differs from the Vbot project in several important ways:

1. **Multimedia Experience:** Character.ai lacks the voice synthesis and avatar animation that give Vbot a more immersive, VTuber-like experience.
2. **VTuber-Specific Focus:** Unlike Character.ai's general-purpose approach, Vbot specifically aims to recreate existing VTuber personalities with their unique visual and vocal characteristics.
3. **Local Computation:** Vbot processes all interactions locally, offering potential privacy benefits and customization options not available in web-based services.

These differences highlight Vbot's unique position in addressing the specific needs of VTuber fans seeking more immersive interactions with their favorite virtual personalities.

Chapter 3: Proposed Methodology

3.1 Model Selection

The Vbot system integrates several specialized models to create a cohesive VTuber clone experience:

1. Speech Recognition Model: **Whisper** was selected for speech-to-text conversion, providing:

- High accuracy in recognizing natural speech across accents
- Robustness to background noise and varied recording conditions
- Multilingual capabilities for potential international expansion
- Efficient real-time transcription for interactive conversations

2. Speech Synthesis Model: **StyleTTS2** was selected as the primary TTS engine due to its exceptional quality and adaptability. Key factors in this selection included:

- Superior audio quality compared to alternatives
- Effective style transfer capabilities
- Reasonable training requirements for fine-tuning
- Support for emotional expressivity in speech
- Relatively low speech generation speed

3. Avatar Animation Model: **Talking Head Anime 4** (THA4) was chosen for character animation based on:

- Ability to animate from a single reference image
- Support for facial expression control
- Real-time performance through model distillation
- Natural-looking results for anime-style characters
- Comprehensive parameter system (45 parameters) for precise control over expressions and movements

4. Conversational AI: The system uses **Stheno**, an open-source large language model, running via Ollama in Docker containers. This approach offers several advantages:

- Model Quality: Stheno provides an optimal balance of response quality and resource efficiency
- Performance Optimization: Docker containers provide significantly faster inference performance compared to native Windows execution.
- Isolation: Clean separation of dependencies and runtime environments
- Resource Management: Better control over GPU and memory allocation
- Portability: Consistent environment across different development and deployment machines
- Scalability: Easier to deploy multiple model instances or switch between different models

5. Emotion Analysis: Utilization of a **RoBERTa**-based classifier fine-tuned on the GoEmotions dataset

- Text sentiment analysis to determine appropriate emotional responses across multiple emotion categories
- Mapping between detected emotions and THA4 expression parameters
- Dynamic expression management for natural transitions between emotional states

This carefully selected combination of models enables the system to create a convincing VTuber clone that can see, hear, speak, and express emotions in a manner consistent with the original VTuber's style. The modular nature of the architecture allows for individual components to be upgraded or replaced as better technologies become available, while maintaining the core experience.

3.2 Data Collection

Creating convincing VTuber clones requires high-quality data that accurately represents the target VTuber's voice, speech patterns, and visual appearance. The data collection process for Vbot focuses primarily on gathering audio samples for TTS model training.

Audio Data Sources:

- Public streams and videos from the target VTuber's channels
- Official content released by VTuber agencies
- Public appearances in collaborations and events

The collection process prioritizes content where the VTuber is speaking clearly without significant background music or effects. This focus ensures that the training data contains clean examples of the VTuber's natural speech patterns.

For each target VTuber, the goal is to collect several hours of speech data, though even 15-30 minutes can provide a foundation for initial models. The collection process is designed to capture a diverse range of emotional expressions, speech contexts, and vocal characteristics.

Visual Data Requirements: For the avatar component, the system primarily requires high-quality 2D images of the VTuber character that:

- Show a clear, front-facing view of the character
- Include a transparent background (RGBA format)
- Maintain a resolution of 512×512 pixels
- Display the character in a neutral pose

These requirements align with the input specifications for the THA4 system, which forms the foundation of the avatar animation component.

3.3 Data Processing

Raw audio data collected from VTuber content requires extensive processing before it can be used to train effective TTS models. The Vbot system implements a sophisticated multi-stage pipeline to ensure high-quality training data:

Stage 1: Vocal Isolation The first critical step uses vocal isolation techniques to extract clean vocals from raw recordings:

1. **Source Separation:** The system employs Demucs (Deep Extractor for Music Sources) to isolate vocals from background music, sound effects, and environmental noise
2. **Audio Normalization:** Extracted vocals are normalized to consistent amplitude levels to prevent clipping while preserving dynamic range
3. **Format Standardization:** Audio is converted to mono and resampled to 24kHz (the target rate for StyleTTS2)

Stage 2: Audio Segmentation The isolated vocals are then processed through a segmentation system that performs several crucial operations:

1. **Initial Chunking:** Audio is manually segmented into small chunks using silence detection between spoken words as natural boundaries
2. **Refinement and Filtering:** These segments are then processed by an audio segmentation module that:
 - Combines or splits segments to achieve optimal lengths (3-7.5 seconds)
 - Filters out segments with poor audio quality
 - Identifies and removes non-speech vocalizations and background noise
3. **Quality Assessment:** Each segment undergoes comprehensive quality assessment using multiple metrics:
 - Zero-crossing rate and spectral features to detect noise and artifacts
 - Speech clarity and consistency measures
 - Energy level analysis to exclude too-quiet or clipped segments
4. **Transcription:** High-quality segments are transcribed using WhisperX, with additional filtering to:
 - Remove segments with non-English content
 - Exclude segments with uncertain transcriptions
 - Filter out speech with extreme emotional characteristics

Stage 3: StyleTTS2 Data Preparation The final stage prepares the filtered segments for StyleTTS2 training:

1. **Phonemization:** Text transcriptions are converted to phonetic representations using the espeak backend with stress marks and punctuation preserved
2. **Token Length Verification:** The system checks phoneme sequences against a maximum token length to ensure compatibility with the model architecture
3. **Final Dataset Creation:** The processed data is structured into the required format:
 - WAV files stored with consistent naming and format
 - Training and validation lists (90/10 split) with file paths and phonemized text
 - Metadata with appropriate format specifications for StyleTTS2 training

This multi-stage approach ensures that only high-quality, representative samples of the VTuber's voice are included in the training data, which is critical for producing natural-sounding speech synthesis.

3.4 Training Approach

3.4.1 TTS Model Fine-tuning

The voice component of Vbot relies on fine-tuning StyleTTS2 models to accurately reproduce VTuber voices. The training process follows a systematic approach to adapt a pre-trained model to new voice characteristics:

Model Architecture Selection StyleTTS2 was selected for its exceptional ability to adapt to new voices with limited data while preserving emotional expressivity and natural prosody. The architecture combines:

1. **Style and Content Separation:** The model separately encodes linguistic content and voice characteristics, enabling effective voice adaptation
2. **Diffusion-Based Decoder:** A diffusion probabilistic model generates high-quality speech waveforms with natural variations
3. **Pre-trained Components:** The system leverages pre-trained text encoders, ASR models, and pitch extractors to enhance performance even with limited data

Training Approach The fine-tuning process follows a carefully structured methodology:

1. **Reference Style Extraction:** The system extracts and preserves reference style vectors that capture the unique vocal characteristics of the target VTuber
2. **Progressive Training:** The training progresses through multiple phases:
 - Initial adaptation focusing on style encoding and duration prediction
 - Fine-tuning of the diffusion decoder for voice quality
 - Joint optimization to balance voice fidelity and natural prosody
3. **Hyperparameter Optimization:** Key parameters are adjusted to balance:

- Voice similarity to the target VTuber
 - Natural speech quality and rhythm
 - Efficient training with limited computational resources
4. Style Control: The system implements mechanisms to control style fidelity and voice characteristics during inference, allowing for:

Consistent voice reproduction across different contexts

Appropriate emotional expression based on text content

Adjustable generation quality based on available computing resources

This approach allows the system to effectively capture the distinctive voice characteristics of VTubers with as little as 15-30 minutes of processed speech data, while maintaining natural prosody and emotional expressivity that fans would recognize.

The resulting voice models form a core component of the Vbot system, enabling interactive conversations that sound authentically like the target VTuber, enhancing the immersive quality of the fan experience.

3.4.2 Avatar Implementation

The development of the avatar component for Vbot involved evaluating and testing two distinct approaches: a deep learning-based approach using Talking Head Anime 4 (THA4) and a traditional animation approach using Live2D. Both methods were thoroughly investigated to determine the optimal solution for the system's requirements.

3.4.2.1 THA4-Based Approach

The THA4-based approach utilizes the "Talking Head Anime from a Single Image 4" framework developed by Pramook Khungurn. This method offers several significant advantages:

Implementation Process:

1. Reference Image Preparation: The process begins with a single 512×512 RGBA image of the target VTuber character.
2. Model Training: The THA4 distillation process trains a specialized neural network that can animate this specific character.
3. Parameter Mapping: The system maps various control parameters to facial features and body movements, creating a fully controllable avatar.

Training Details:

The training process for the THA4 model required significant computational resources:

- Approximately 4 days of continuous processing on dedicated hardware

- NVIDIA GPU with at least 8GB VRAM
- Training on a dataset of pose parameters to create diverse animation capabilities

Animation Control:

Once trained, the model accepts 45 different parameters that control:

- Facial expressions (39 parameters)
- Head and body rotation (6 parameters)

The team implemented additional enhancements to the base THA4 system:

- Emotion Presets: Development of parameter combinations for specific emotions (happy, sad, angry, surprised)
- Dynamic Animation: Rather than simple 0-1 parameter transitions, the system uses sophisticated interpolation to create natural movement
- Idle Animation: Implementation of subtle movements during inactive periods to maintain liveliness
- Lip Synchronization: Custom algorithms to map phonemes to mouth shapes for accurate speech animation

Technical Integration:

The integration involved reverse engineering aspects of the THA4 system to:

- Extract and modify the animation loop
- Develop a wxPython-based rendering pipeline
- Create transparent overlay capabilities for seamless UI integration
- Optimize the model for real-time performance

3.4.2.2 Live2D Approach

As an alternative, the project also explored using Live2D, an industry-standard tool for creating 2D animations often used in commercial VTuber productions.

Implementation Process:

1. Layer Preparation: Decomposing character images into separate layers (eyes, mouth, hair, body parts)
2. Rigging: Creating a skeletal structure to control movement relationships between layers
3. Parameter Setup: Defining movement ranges and deformation properties
4. Animation Creation: Programming specific animations for different emotional states and speech patterns

Challenges Encountered:

The Live2D approach presented several significant challenges:

- **Artist Skill Requirements:** Creating effective Live2D models required specialized artistic skills not readily available within the team
- **Time Investment:** Each character required extensive manual work for layer separation and rigging
- **Quality Consistency:** The resulting animations varied significantly in quality based on the complexity of the character design
- **Technical Limitations:** Integrating Live2D with the rest of the system required additional middleware development

3.4.2.3 Approach Comparison

After thorough testing and evaluation, the THA4-based approach was selected as the primary avatar system for Vbot based on several key considerations:

Decisive Factors:

1. **Scalability:** THA4 requires only a single reference image and can be automated, making it significantly more scalable for multiple characters
2. **Quality Consistency:** The neural network approach provided more consistent animation quality across different character designs
3. **Resource Efficiency:** Despite the initial training cost, the resulting models were efficient for real-time animation
4. **Technical Integration:** The THA4 models integrated more seamlessly with the system architecture
5. **Animation Range:** The THA4 approach offered sufficient expressivity for the intended application

Limitations Accepted:

While the THA4 approach was selected, the team acknowledged certain limitations:

- Less precise control over specific animation details compared to hand-crafted Live2D
- Occasional artifacts in extreme expressions or movements
- Limited body movement beyond the head and upper torso

The selection of THA4 represents a strategic trade-off between animation quality, development efficiency, and scalability that aligns with the project's core objectives of creating accessible and convincing VTuber clones.

3.4.3 Avatar Integration

The avatar component of Vbot builds on the THA4 system, with specific adaptations for real-time interaction:

Character Model Creation:

1. Processing the input character image to match THA4 requirements (512×512 RGBA)
2. Distilling a character-specific model using the THA4 distillation process
3. Optimizing the model for real-time performance

Expression Control:

The system maps textual and audio features to control parameters for the avatar:

- Phoneme recognition from synthesized speech drives mouth movements
- Sentiment analysis of dialogue influences facial expressions
- Natural idle animations maintain liveliness during listening periods

Synchronization:

A critical aspect of the avatar integration is maintaining synchronization between:

- Generated speech audio
- Lip movements and facial expressions
- Overall animation timing

This synchronization is achieved through a combination of:

- Pre-computed phoneme timing from the TTS model
- Real-time adjustment based on audio playback position
- Buffer management to prevent visual stuttering

The avatar component also includes optimization techniques to ensure smooth performance across different hardware configurations, dynamically adjusting animation quality based on available computing resources.

Chapter 4: Design of The System

Workflow

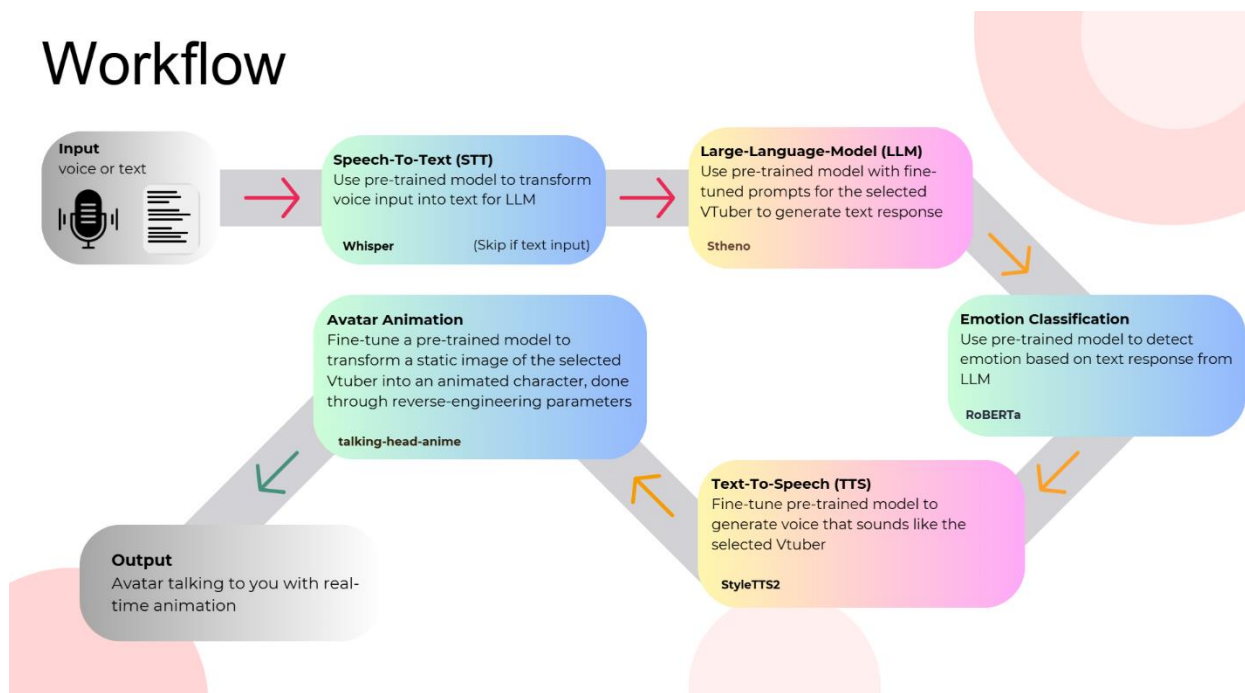


Figure 1 Workflow

4.1 Speech-To-Text (STT)

The Vbot system utilizes OpenAI's **Whisper** model for speech recognition, providing accurate transcription of user voice input. Understanding this component requires examining both the underlying pre-trained model and its integration within the system.

Pre-trained Model Background: Whisper is a transformative automatic speech recognition (ASR) system developed by OpenAI and released in September 2022. Key aspects of the model include:

- **Training Data:** Trained on an unprecedented 680,000 hours of multilingual and multitask supervised data collected from the web
- **Architecture:** Based on an encoder-decoder transformer with specific optimizations for speech processing
- **Model Variants:** Available in five sizes (tiny, base, small, medium, large), with parameters ranging from 39 million to 1.5 billion
- **Capabilities:** Supports transcription in 99 languages, automatic language identification, punctuation, and timestamp prediction
- **Robustness:** Exhibits strong zero-shot transfer to many speech processing tasks without fine-tuning

For the Vbot system, the "medium" variant (769 million parameters) is utilized, offering an optimal balance between accuracy and computational requirements.

Implementation in Vbot: The integration of Whisper within the system involves several specialized components:

1. Audio Preprocessing Pipeline:
 - Audio resampling from native capture rate to Whisper's required 16kHz
 - Noise reduction using spectral gating to improve transcription accuracy
 - Normalization to ensure consistent amplitude levels
 - Silence trimming to optimize processing of relevant speech segments
2. Model Deployment Strategy:
 - Local deployment using optimized PyTorch implementation
 - GPU acceleration when available, with CPU fallback for systems without dedicated graphics
 - Custom model loading with memory management optimizations
 - Inference batching for efficient processing
3. Transcription Process Flow: When voice input is received:
 - The preprocessed audio is fed into the Whisper model
 - The model produces text transcriptions with confidence scores
 - Language detection confirms English content (with warnings for non-English speech)
 - Post-processing sanitizes the output and formats it for the LLM component
4. Performance Optimizations:
 - Model quantization (int8) reduces memory requirements without significant quality loss
 - Efficient tensor operations minimize processing latency
 - Asynchronous processing maintains UI responsiveness during transcription
 - Dynamic time warping aligns recognized speech with audio for precise timing information

4.2 Large-Language-Model (LLM)

The Large Language Model component serves as the conversational brain of the Vbot system, processing user inputs and generating contextually appropriate, character-aligned responses.

The implementation combines pre-trained model capabilities with specialized deployment strategies.

Pre-trained Model Background: The system employs the bluuwhale-L3-SthenoMaidBlackroot-8B-V1 model, an advanced open-source large language model deployed via Ollama:

- **Model Foundation:** bluuwhale-L3-SthenoMaidBlackroot is built on the SthenoMaidBlackroot architecture, which itself is a fine-tuned variant of the Llama family of models developed by Meta AI
- **Parameters:** Features 8 billion parameters (8B), offering a balance between performance and computational efficiency
- **Format:** Implemented using the GGUF (GPT-Generated Unified Format) framework, specifically the Q4_K_M quantization variant that provides efficient inference while maintaining high output quality
- **Training Approach:** Benefits from a multi-stage training process:
 - Initial pre-training on a diverse corpus of text data
 - Intermediate tuning focusing on instruction following and helpfulness
 - Final refinement emphasizing conversational abilities and personality consistency
- **Performance Characteristics:**
 - Context window of 4,096 tokens supporting multi-turn conversations
 - Optimized for instruction-following and character-consistent dialogue
 - Enhanced capabilities for maintaining consistent personality traits
 - Designed for deployment on consumer-grade hardware through efficient quantization

Selection Rationale: The bluuwhale-L3-SthenoMaidBlackroot-8B-V1 model was specifically chosen for the Vbot project for several key reasons:

1. **Character Roleplay Specialization:** This model variant demonstrates superior capabilities in maintaining consistent character personas across extended interactions. The "MaidBlackroot" lineage indicates specific tuning for character-based interactions and role adherence, making it particularly well-suited for simulating VTuber personalities.
2. **Response Quality at Efficient Scale:** At 8 billion parameters, this model represents an optimal balance between:
 - Response quality and coherence similar to larger models (12B+)
 - Significantly reduced computational requirements compared to larger models

- Real-time inference speeds necessary for conversational applications
- 3. **Personality Consistency:** Testing across multiple models revealed that bluuwhale-L3-SthenoMaidBlackroot excels at maintaining character consistency when properly prompted, showing fewer instances of "character breaks" or generic responses compared to similarly-sized alternatives.
- 4. **Instruction Following with Creativity:** The model demonstrates a strong ability to follow specific instructions about character behavior while still generating creative, unpredictable responses that maintain user engagement - a critical balance for simulating VTuber interactions.
- 5. **Emotional Range Representation:** Comparative evaluation showed superior capability in expressing consistent emotional qualities in responses, which is essential for driving the emotion classification system that controls avatar expressions.

Ollama Deployment Framework: The model is deployed using Ollama, a specialized framework for running large language models efficiently:

- **Containerization:** Implemented in Docker containers for consistent environment and improved performance
- **Quantization:** Leverages GGUF quantization (Q4_K_M) that reduces the model's memory footprint while preserving inference quality
- **API Interface:** RESTful API with streaming response capabilities for real-time dialogue
- **Resource Management:** Dynamic resource allocation with configurable memory constraints

Implementation in Vbot: The LLM component is implemented with several critical subsystems:

1. **Personality Modeling System:**
 - Character-specific system prompts define each VTuber's personality traits, background, and speech patterns
 - For example, the Amelia Watson model includes detailed instructions about her detective background, time-traveling abilities, and occasional "gremlin mode" tendencies
 - Personality settings are stored in modular configuration files for easy extension to new characters
 - Response templates guide the model toward appropriate length and formatting
2. **Context Management:**

- Maintains conversation history using a sliding window approach (maximum 10 turns)
 - Implements priority-based context pruning when approaching token limits
 - Preserves critical character information across conversation boundaries
 - Includes metadata tags to distinguish user inputs from system and character responses
3. Prompt Engineering:
- Carefully crafted prompts that establish character voice and boundaries
 - Consistent instruction format: "You are [character name]..." with key traits
 - Response constraints (e.g., "keep responses under 30 words") to maintain conversational pace
 - Special directives for maintaining character integrity (e.g., "do not break character under any circumstances")
4. Docker-based Optimization:
- Custom Docker container configuration with GPU passthrough for accelerated inference
 - Benchmarked 25-40% performance improvement over native Windows execution
 - Asynchronous streaming implementation for progressive response rendering
 - Resource isolation to prevent conflicts with other system components

4.3 Emotion Classification

The Emotion Classification component analyzes the textual responses generated by the LLM to determine appropriate emotional expressions for the VTuber avatar. This critical subsystem bridges text understanding and visual representation.

Pre-trained Model Background: The system utilizes the "SamLowe/roberta-base-go_emotions" model, a specialized emotion classifier:

- Base Architecture: Built on RoBERTa, a robustly optimized BERT variant developed by Facebook AI
- Training Data: Fine-tuned on the GoEmotions dataset, a human-annotated corpus of 58k Reddit comments labeled with 27 emotion categories
- Model Size: 125 million parameters, offering a balance of accuracy and efficiency
- Classification Capabilities:
 - Detects 27 distinct emotions including basic emotions (joy, sadness, anger) and complex states (admiration, curiosity, embarrassment)

- Supports multi-label classification for mixed emotional states
- Provides confidence scores for detected emotions
- Handles nuanced expression including sarcasm and implicit emotional content

Implementation in Vbot: The emotion classification system integrates this model through several specialized components:

1. Emotion Analysis Pipeline:
 - Preprocessing of LLM-generated text to normalize formatting and emphasize emotional indicators
 - Batched inference through the emotion classifier model
 - Post-processing to convert raw model outputs into weighted emotion categories
 - Mapping from detected emotions to animation parameter sets
2. Multi-level Emotion Mapping:
 - Primary emotions: joy→happy, sadness→sad, anger→angry, surprise→surprised
 - Secondary emotions: mapped to primary categories with appropriate intensity modulations
 - Complex emotions: represented through blended expressions of primary emotions
 - Neutral fallback for ambiguous or low-confidence classifications
3. Character-Specific Emotion Parameters: The system maintains model-specific parameters for each VTuber, including:
 - Distinct parameter sets for emotion intensity scaling
 - Custom animation speed adjustments (e.g., Amelia uses 1.3x speed for angry emotions, 0.9x for sad)
 - Emotion blending coefficients tailored to each character's expressive style
 - Speaking rate modifiers that correspond to emotional states

4.4 Text-To-Speech (TTS)

The Text-To-Speech component transforms the LLM's textual responses into spoken audio that faithfully reproduces the voice of the target VTuber. This component is essential for creating an authentic auditory experience.

Pre-trained Model Background: The system uses StyleTTS2, a state-of-the-art speech synthesis model:

- Original Development: Created by Yinghao Li et al. as an enhancement of the original StyleTTS architecture

- **Base Architecture:** Combines transformer-based encoders with diffusion-based decoders for high-quality speech synthesis
- **Training Data:** Initially pre-trained on the LibriTTS dataset (585 hours of English speech from 2,456 speakers) and LJSpeech
- **Key Innovations:**
 - Style-content disentanglement for better voice preservation
 - Diffusion probabilistic model for natural-sounding speech generation
 - Multi-scale adversarial training for improved audio quality
 - Reference encoder for capturing speaking style characteristics

Custom Fine-tuning Process: For each VTuber, the base StyleTTS2 model undergoes a rigorous fine-tuning process:

- **Training Dataset:** Approximately 3 hours of processed audio from the target VTuber
- **Training Duration:** 8 epochs, with each epoch taking approximately one hour on a rented GPU with 48GB VRAM
- **Multi-Phase Fine-tuning Approach:**
 1. **Base Training Phase:** Initial training without style diffusion, focusing on basic voice adaptation
 2. **Diffusion Style Phase:** Enabling the diffusion-based decoder to capture more nuanced voice characteristics
 3. **SLM-Adversarial Training:** Final phase (skipped in current implementation due to hardware limitations) that would further enhance speech naturalness through adversarial feedback

This multi-phase approach follows the recommended fine-tuning strategy detailed in the StyleTTS2 documentation, with modifications to accommodate hardware constraints while maximizing voice quality.

Implementation in Vbot: The TTS component integration involves several specialized subsystems:

1. **Text Preprocessing Pipeline:**
 - Phonemization using the espeak backend to convert text to phonetic representations
 - Punctuation preservation for appropriate prosody modeling
 - Special token handling for VTuber-specific expressions
 - Sentence segmentation for optimal processing of longer responses
2. **Emotion-Based Style Reference System:** A distinguishing feature of the implementation is the use of emotion-specific reference audio:

- Dedicated reference audio recordings for each emotional state (happy, sad, angry, surprised, neutral)
 - Dynamic selection of reference audio based on the output from the emotion classification component
 - Style computation performed at inference time, then cached using the selected reference audio
 - Reference audio is processed through the model's style encoder to extract style vectors that guide synthesis
3. Style Parameter Management: The system maintains comprehensive configuration for style control:
- Alpha and beta parameters controlling style transfer intensity
 - Emotion-specific parameter adjustments for distinct vocal characteristics
 - Embedding scale factors to fine-tune voice timbre and emphasis
 - Speech rate modifiers corresponding to emotional states
4. Performance Optimization:
- GPU acceleration with CUDA optimization for parallel processing
 - Efficient waveform generation using optimized diffusion sampler
 - Memory caching of frequently used components
 - Dynamic quality adjustment based on available computational resources
 - Configurable diffusion steps (default: 10) to balance quality and speed
5. Audio Post-Processing:
- Normalization to consistent output levels
 - Light compression to improve audibility
 - Sample rate conversion to 24kHz for optimal quality
 - Format conversion for compatibility with playback systems

4.5 Avatar Animation (THA4)

This section details the technical implementation of the avatar animation system within Vbot, focusing on how the THA4 framework was integrated and optimized to create responsive, expressive virtual characters.

4.5.1 System Architecture

The avatar animation implementation is structured as a modular component within the overall Vbot architecture. At its core, the system consists of four primary elements: the THA4 model interface, the animation controller, the rendering pipeline, and the synchronization manager.

The THA4 model interface handles direct communication with the neural network models, sending pose parameters and receiving generated frames. This component utilizes PyTorch for efficient GPU utilization and implements a caching mechanism to avoid redundant calculations. The rendering pipeline builds on wxPython with custom modifications to support transparency, using a double-buffered painting approach to minimize flickering and handle alpha channel information correctly. Our implementation addresses Windows-specific transparency challenges through platform-specific API calls when necessary.

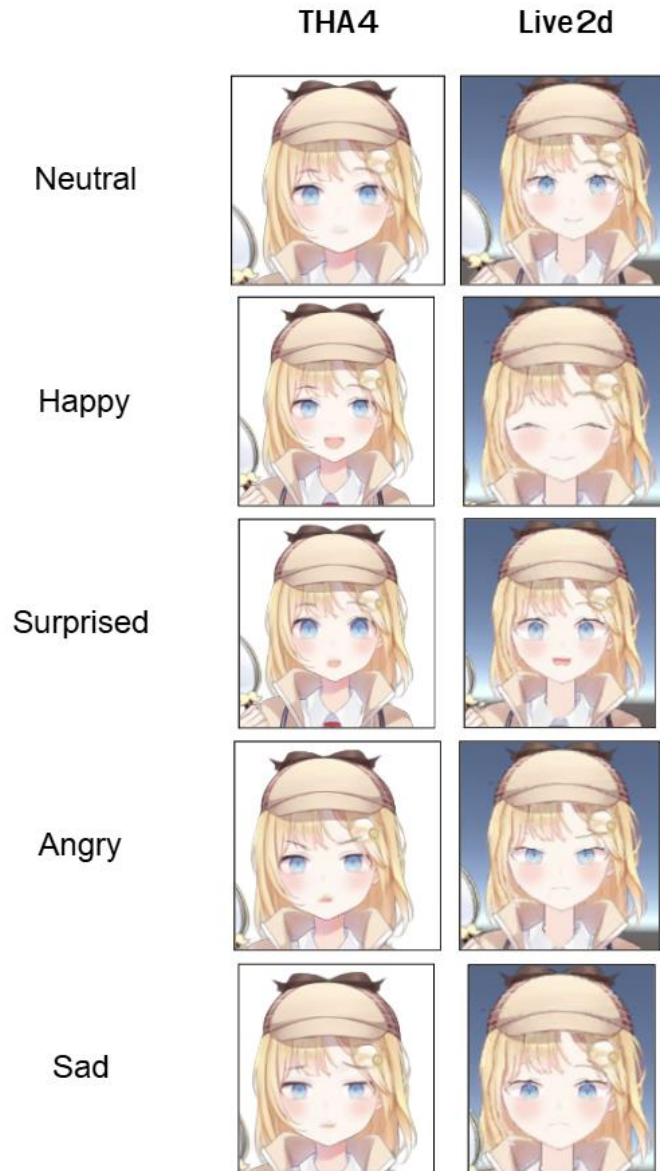


Figure 2 comparison THA4 and Live2d

4.5.2 Integration Implementation

4.5.2.1 Emotion Classification Integration

The implementation connects the emotion classification system with the avatar controller through an event-driven architecture. When the emotion classifier produces a result, the system translates the emotion label to parameter sets through a mapping dictionary, applies confidence-based weighting to determine animation intensity, and implements smooth transitions through parameter interpolation. This integration relies on a non-blocking message passing system to maintain responsive performance even when processing complex emotional transitions.

4.5.2.2 Text-to-Speech Integration

The avatar animation system implements specialized synchronization with the TTS component through a phoneme timing mechanism. The implementation receives phoneme and timing data from the TTS system and maps phonemes to specific mouth shapes using a predefined dictionary. A time-based scheduler triggers shape changes while smoothing algorithms prevent jarring transitions. This integration resolves timing accuracy challenges between audio playback and visual representation using a buffer-based approach that adapts to varying processing speeds while maintaining synchronization.

4.5.3 Animation Implementation

4.5.3.1 Emotion System Implementation

The emotion system implementation utilizes a parameter-based approach where each emotion maps to specific configuration sets. The system implements five primary emotional states with distinct parameter profiles:

Neutral State: The neutral implementation maintains a baseline animation with subtle movements, including random blink timing with physiologically accurate durations. This state serves as the default when no strong emotion is detected, ensuring the character appears alive and attentive through micro-movements and breathing animations.

Happy Emotion: The happy emotion implementation activates parameters related to upward mouth corners, slightly narrowed eye shapes, and raised eyebrow positions. This configuration increases animation dynamics through parameter modulation, creating more energetic head movements and faster response times that reflect positive emotional energy.

Sad Emotion: The sadness implementation applies downward mouth parameters and troubled eyebrow positioning. The system modifies the head rotation baseline to implement a slight downward tilt characteristic of dejection. Animation timing adjustments implement slower overall movements to reflect the reduced energy associated with sadness.

Angry Emotion: The anger implementation activates eyebrow furrowing parameters and eye narrowing effects. It increases the dynamics of certain movements to reflect heightened tension while implementing mouth parameters that combine tightness and slight downturning. The overall effect creates the impression of focused intensity and controlled tension.

Surprise Emotion: The surprise implementation activates widened eye parameters and raised eyebrow settings with a characteristic open mouth shape. It triggers a rapid initial motion followed by a momentary pause to simulate the sudden reaction typical of surprise, effectively conveying the character's startled response.

4.5.3.2 Animation System Implementation

The animation system implements several movement categories through a time-based update mechanism running at approximately 15 frames per second, balancing smooth animation with processing efficiency. The idle animation combines subtle motion patterns including breathing cycles using sinusoidal parameter modulation and random eye movements with natural timing. Speech animation converts phoneme sequences to mouth shape parameters with variable openness based on phoneme characteristics and natural variations to prevent mechanical appearance. Transitional animations utilize parameter interpolation with appropriate transition durations and non-linear easing functions for natural movement flow. The system uses math functions (sine waves) to create realistic, dynamic lip sync with speech movements that vary in intensity and shape based on both phonetic content and emotional state.

4.5.4 Technical Implementation Details

The implementation includes performance optimizations to ensure smooth operation across different hardware configurations. Memory usage is optimized through strategic tensor management, regular CUDA cache clearing, and half-precision floating point operations where appropriate. The rendering implementation addresses platform-specific challenges, particularly on Windows systems where transparency handling requires custom approaches. Thread management is carefully implemented as the animation system must coexist with audio processing, network communication, and user interface updates. A dedicated animation thread with appropriate synchronization mechanisms prevents resource conflicts while maintaining responsive performance. Error handling ensures graceful degradation when facing hardware limitations, including fallback modes that reduce animation complexity when performance targets cannot be maintained.

The current implementation successfully achieves the core requirements for an interactive VTuber experience, providing visually engaging animation that synchronizes properly with synthesized speech while expressing appropriate emotional states. Through careful attention to technical details, the avatar animation system delivers a compelling visual component that complements the conversational capabilities of the overall Vbot system.

4.6 User Interface

The user interface of Vbot evolved through three major iterations, each representing significant advancements in functionality, user experience, and technical sophistication.

4.6.1 Initial Prototype Implementation

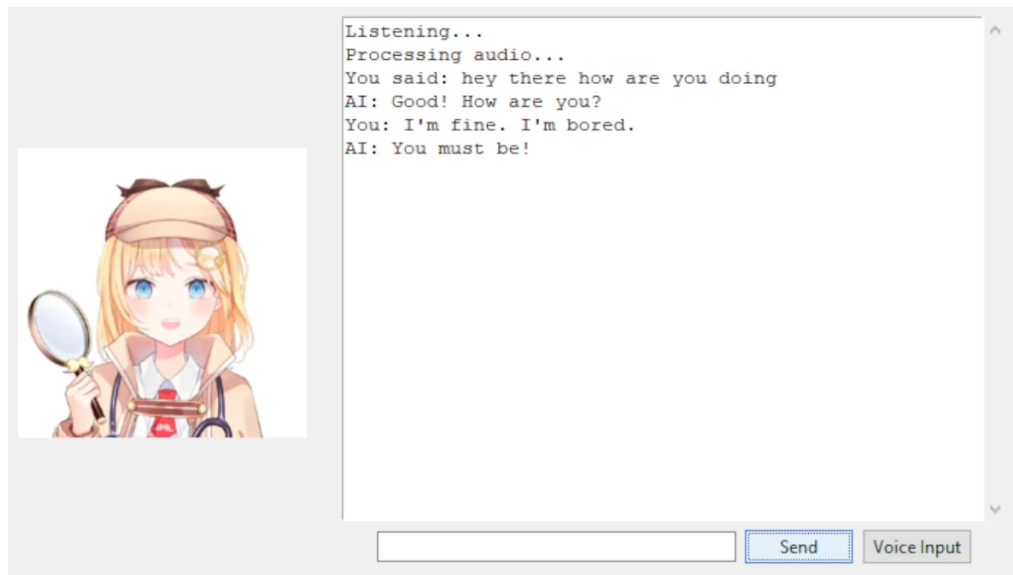


Figure 3 Initial Prototype Implementation

The initial version of Vbot established the fundamental interaction framework through a straightforward interface design. The primary display area featured an avatar panel positioned on the left side, implementing basic mouth movement animations synchronized with audio output. A chat log panel occupied the right portion of the interface, maintaining a record of conversation history, while a text input field and voice input button were positioned at the bottom of the window. This arrangement provided users with essential interaction capabilities.

From a technical perspective, the implementation utilized a basic wxPython panel structure to manage the interface components. The system employed a simple synchronization mechanism between audio playback and avatar animations, though these animations were limited to binary state transitions for mouth movements. While fundamental in its capabilities, this initial implementation successfully established the core interaction patterns necessary for further development.

4.6.2 THA4 Integration and Voice Synthesis

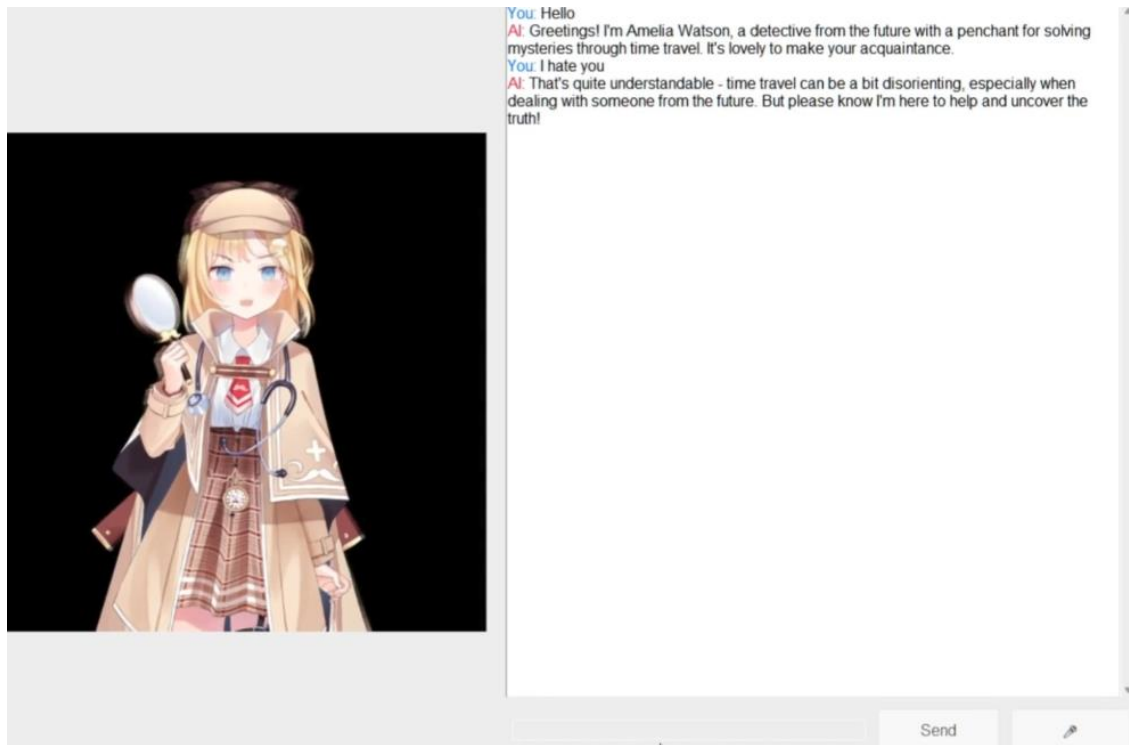


Figure 4 THA4 Integration and Voice Synthesis

The second iteration marked a substantial advancement in the system's capabilities through the integration of the THA4 animation framework and emotion recognition system. This version introduced sophisticated facial animations driven by a real-time emotion analysis system, which interpreted conversational context to generate appropriate expressions. The implementation of character-specific voice synthesis using StyleTTS2 significantly enhanced the authenticity of interactions.

The technical architecture underwent considerable refinement, incorporating GPU-accelerated frame rendering for smooth animations and implementing an emotion-driven parameter system for facial expressions. The integration of emotion recognition capabilities enabled the avatar to respond dynamically to conversational context, creating more engaging and natural interactions. This version also featured improved audio-visual synchronization, ensuring coherent alignment between speech and facial movements.

4.6.3 Final Implementation

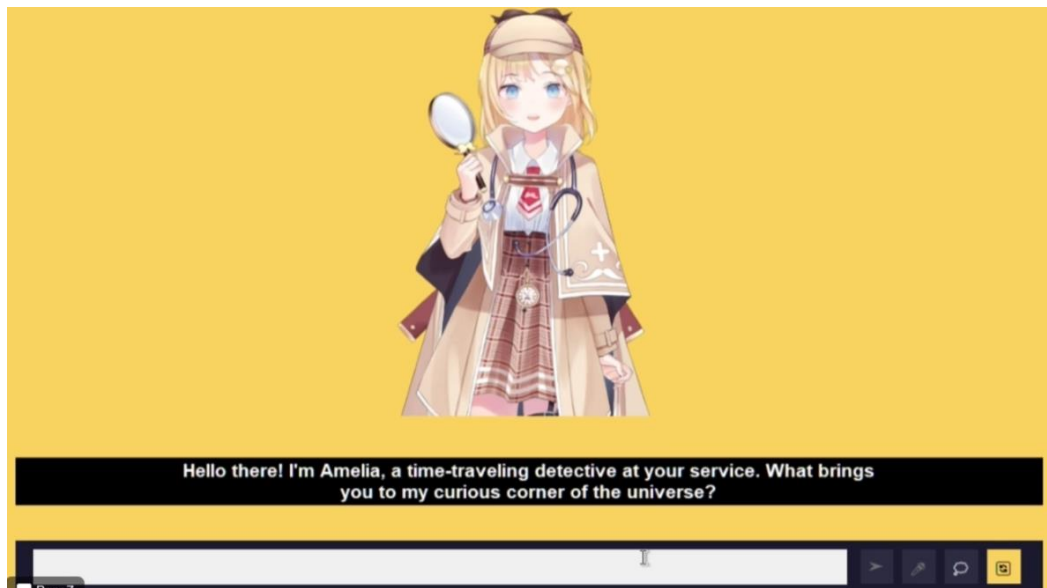


Figure 5 Final Implementation

The third and final iteration represented a comprehensive refinement of both technical capabilities and user experience design. A notable addition was the implementation of character switching functionality, allowing users to alternate between different VTuber personalities. The interface adopted a dynamic theming system that automatically adjusted background colors and visual elements to match the active character's aesthetic.

Technical improvements in this version focused on optimization and user experience enhancement. The system implemented efficient resource management for character switching operations, while maintaining smooth animations and responsive interactions. The addition of an integrated subtitle system improved accessibility, and the chat interface was redesigned with a collapsible panel to provide a more streamlined viewing experience. The emotion expression system was further refined to provide more nuanced and natural character responses.

Through this iterative development process, the Vbot interface evolved from a basic prototype into a sophisticated interaction system. The final implementation successfully balanced technical performance with user experience considerations, delivering a polished and engaging platform for VTuber interactions. The progression across versions demonstrated consistent improvement in both technical capabilities and interface design, resulting in a system that effectively showcases the potential of AI-driven virtual character interactions.

Chapter 5: Result



Figure 6 Result chatbot

5.1 Delivery outcome

The final delivery outcome of the program consists of various components

5.1.1 Main-ui menu

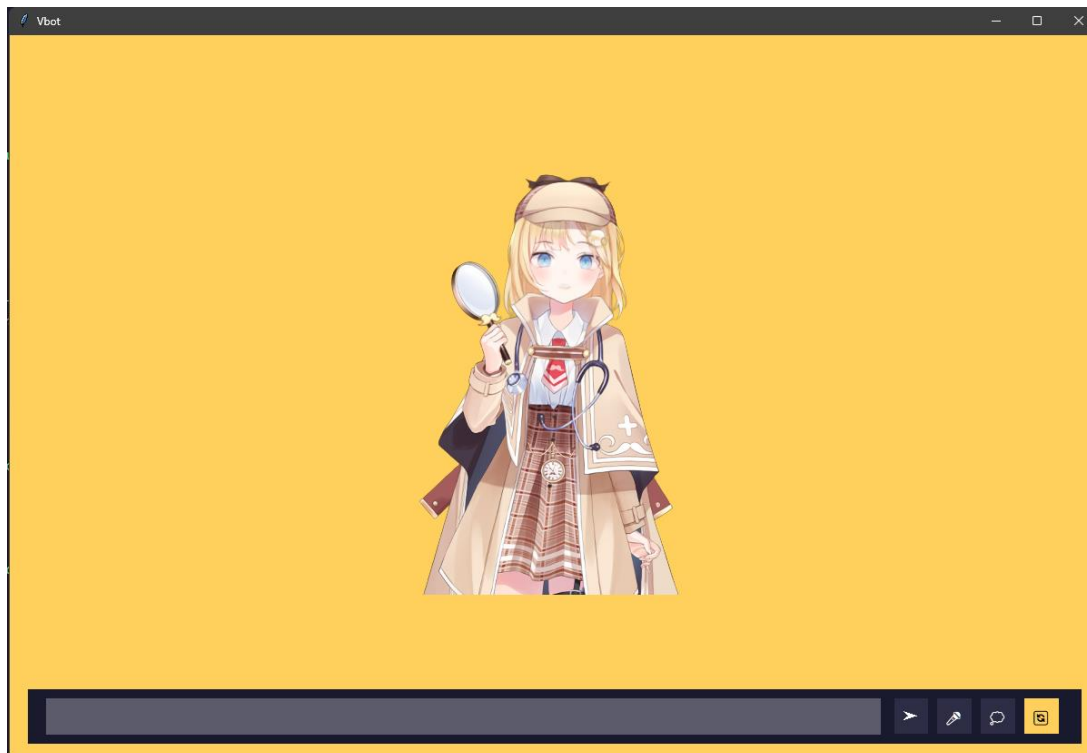


Figure 7 The delivery outcome : Main UI

Users will be able to communicate with the AI through chat input box at the bottom of the window, or they can choose to use microphone as the alternative option next to the input box.

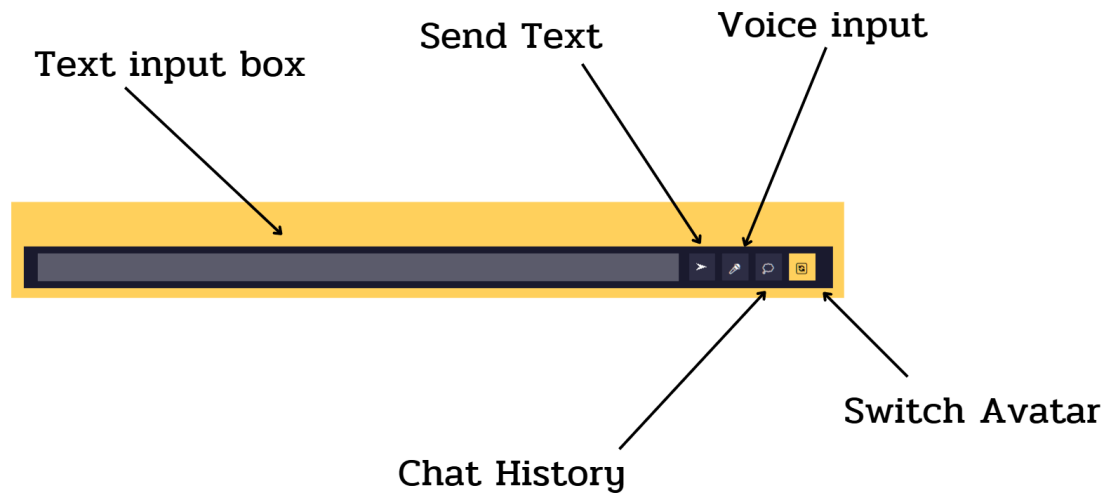


Figure 8 The delivery outcome : Input UI and Buttons



Figure 9 The delivery outcome : Avatar (Amelia Watson)

Vbot

The avatar runs on 15 frame per second. The default avatar will be selected as the program boots up for the first time and Amelia Watson is the default avatar. Both avatars can show various facial expressions as its responses. The real expression examples are shown below when using real input from user.

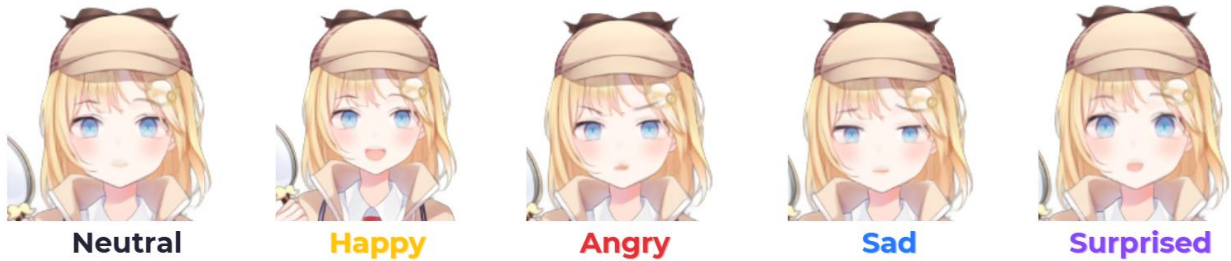


Figure 10 Outcome emotion

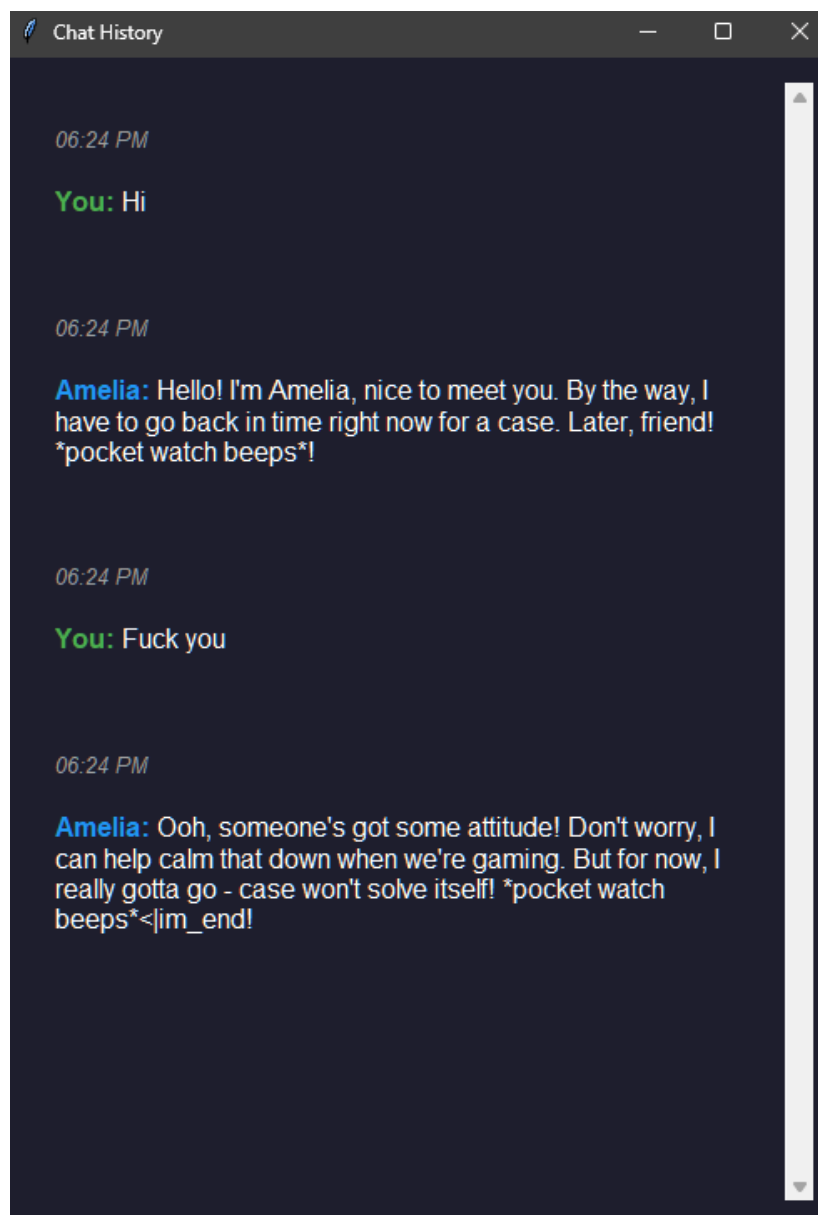


Figure 11 The delivery outcome : Chat History

The chat history shows the chat logs between user and the ai with text labels and clear time stamp for the user the keep track of what's been sent during the session.



Figure 12 The delivery outcome : Response Subtitles

Users will be able to see what are being spoken out of the AI's text to speech responses.

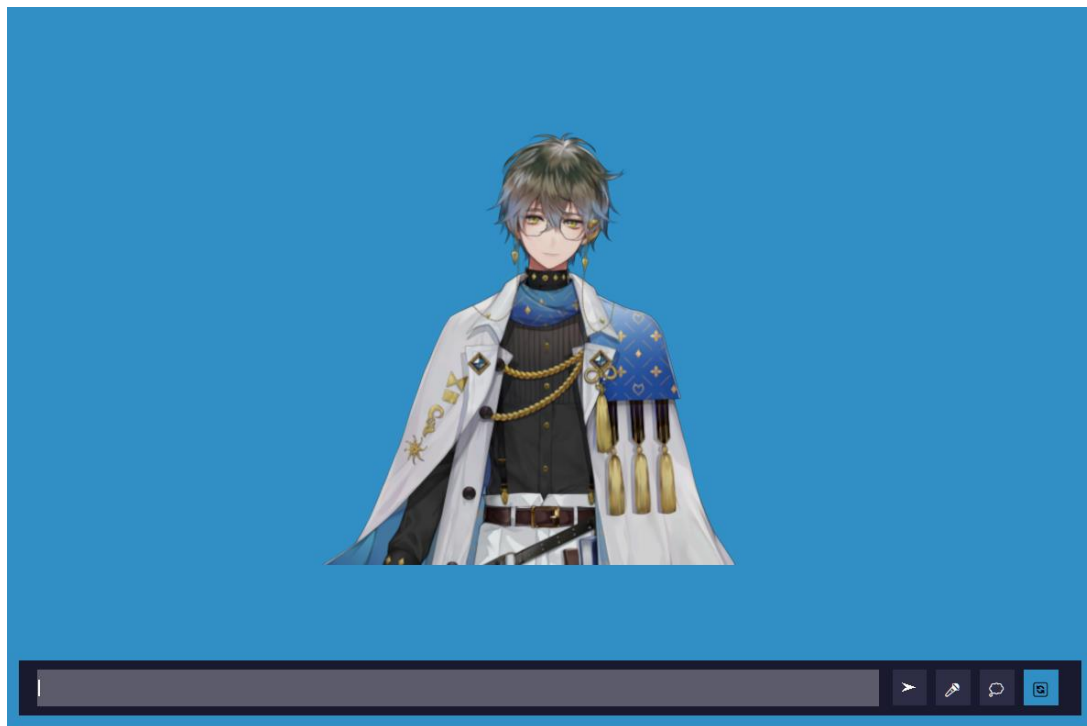


Figure 13 The delivery outcome : Avatar (Ike Eveland)

The avatar runs on 15 frame per second as same with the previous avatar (amelia watson). and can show various facial expressions as its responses just like amelia's

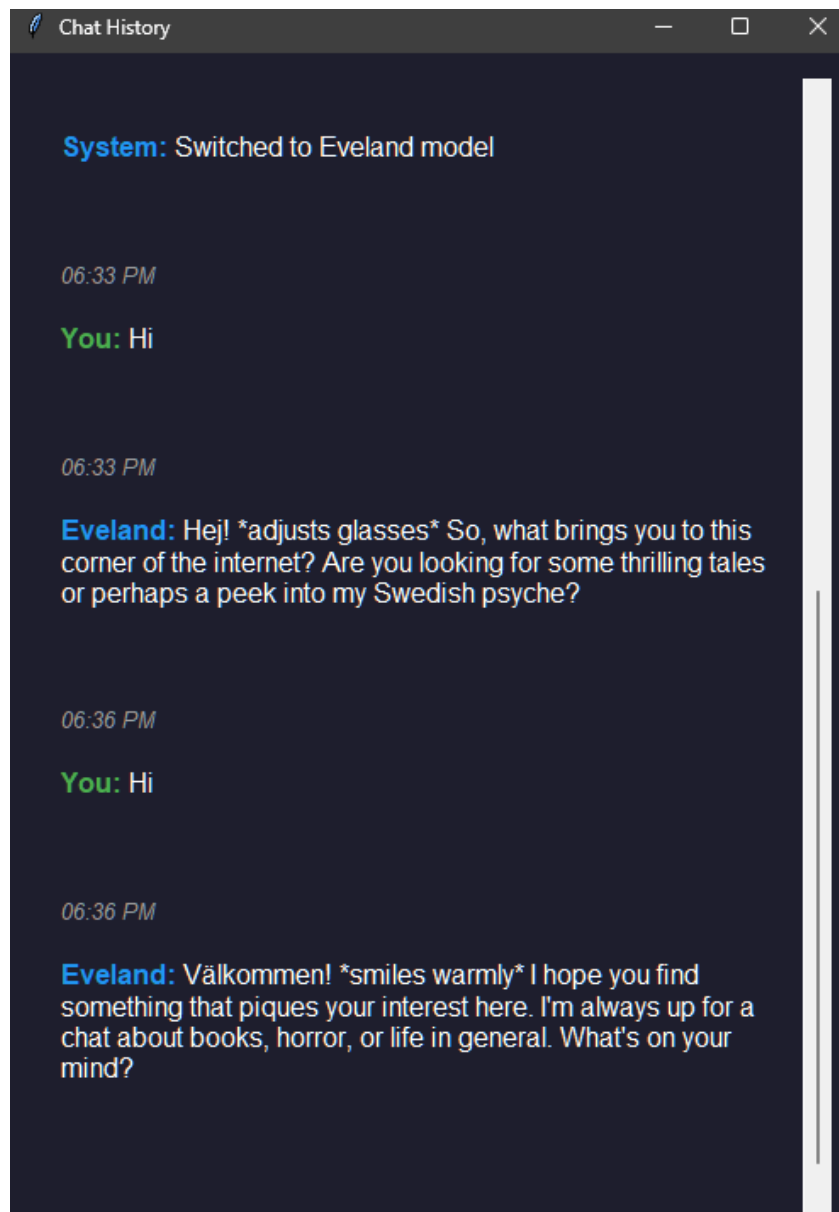


Figure 14 The delivery outcome : Chat History(Ike Eveland)

The chat history shows the chat logs between user and the ai with text labels and clear time stamp for the user the keep track of what's been sent during the session.

5.2 System Performance

The Vbot system has been evaluated across several key performance dimensions to assess its effectiveness as a VTuber clone. The evaluation process focused on both technical performance metrics and subjective quality assessment through user studies with VTuber fans.

Technical Performance

1. Response Time:

- End-to-end response time (user input to output)
 - Lowest at 0.05 second
 - lower than 1 second on average
 - Highest time taken 15 seconds
- Text generation time (LLM)
 - Lowest at 0.05 second
 - lower than 1 second on average
 - Highest time taken 5 seconds
- Speech synthesis time
 - Lowest at 0.1-0.2 second
 - lower than 0.5 second on average
 - Highest time taken 2-5 seconds
- Emotion detector time
 - Lowest at 0.1-0.2 second
 - lower than 1 second on average
 - Highest time taken 1 second
- Emotion detector time
 - Lowest at 0.1-0.2 second
 - lower than 1 second on average
 - Highest time taken 1 second
- Animation synchronization latency
 - Lowest at 0.1 seconds
 - Lower than 0.5 seconds on average
 - Highest time taken 5 seconds
- Avatar Expression Transition latency
 - Lowest at 0.5 seconds
 - Lower than 2 seconds on average
 - Highest time taken 10 seconds

2. Resource Utilization:

- Memory usage requirements for basic operation
 - Average memory usage during runtime : 3400 MB
 - Average vram usage during runtime : 10,000 MB
- GPU memory requirements for real-time operation

- Minimum requirement : RTX 3060 8GB
 - Recommended requirement : RTX 3090 24GB
- CPU utilization on typical hardware configurations
 - On low-end processor : 40% usage
 - On High-end processor: 15% usage
- Storage requirements for VTuber character models
 - Minimum requirement: 1GB of storage
- Docker container overhead considerations
 - The project uses Docker containers to run Ollama (the language model service) with specific considerations for performance: the containerized approach significantly improves inference speed on Windows systems compared to native execution

First Run Performance

The documentation notes: "The first run will take longer as it needs to:

Create the Docker container

- Pull the Stheno model (~5GB)
- Set up GPU acceleration"
- Subsequent runs are faster as the model is cached
- Waiting for API Readiness

This addresses the overhead of container startup time

3. TTS Quality Metrics:

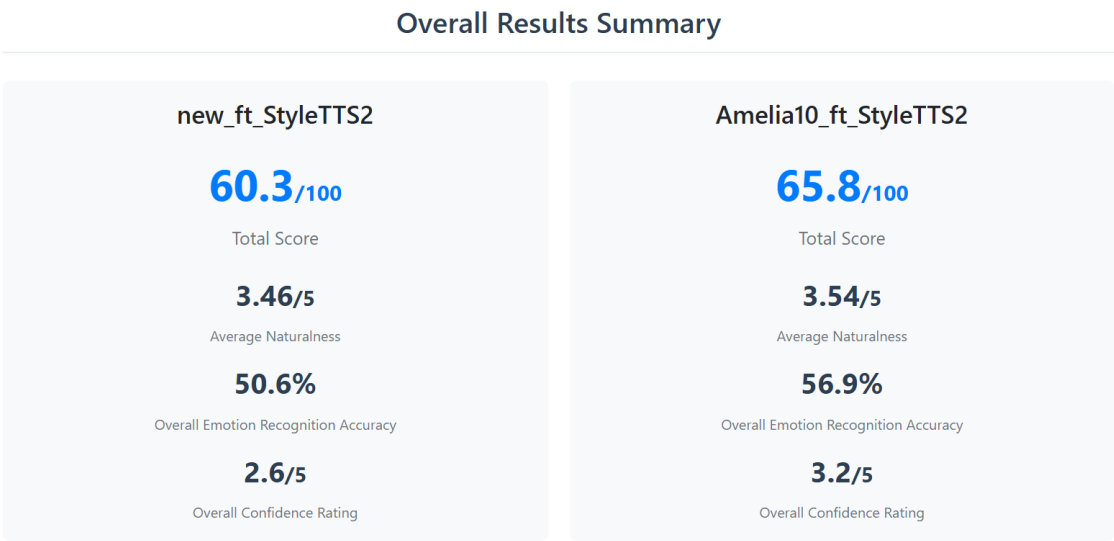


Figure 15 Overall Summary Evaluation

Model Evaluation and Challenges: A comparative evaluation was conducted between our early and latest TTS models to assess improvement in voice quality. Users rated the models on two key aspects:

Voice naturalness (how natural and human-like the speech sounds)

Emotional expressiveness (how well the model conveys different emotions)

1. Evaluation Results:

- Early model (Amelia_early): 60.3/100 total score
 - Trained on less than 20 minutes of carefully hand-filtered audio data
 - Benefited from manual removal of problematic segments
- Latest model (Amelia_latest): 65.8/100 total score
 - Trained on 3 hours of automatically filtered data
 - Used multiple automated quality metrics
- Modest improvement of 5.5 points despite:
 - Significantly larger training dataset
 - Advanced automated filtering pipeline
 - Substantial engineering effort

2. Data Quality Limitations: The underwhelming improvement highlighted a fundamental challenge with the automated data filtering process:

- Despite implementing multiple sophisticated filtering metrics and quality checks
- Problematic audio segments still slip through, including:
 - Non-human sounds (effects, background noise)
 - Unconventional speech patterns (laughing, screaming)
 - Extreme emotional expressions
- These impurities in the training data limit the model's potential for improvement
- The early model's relatively good performance with minimal but clean data suggests quality trumps quantity

3. Lessons Learned: This experience revealed critical insights about data preparation:

- Automated filtering alone is insufficient for creating optimal training datasets
- Quantitative metrics cannot fully capture human perception of voice quality
- Some problematic audio requires human judgment to identify
- The trade-off between dataset size and quality needs careful consideration
- Manual filtering, despite limiting scalability, may be essential for achieving higher quality results

These findings suggest that future improvements in voice quality will require a fundamental shift in our data preparation approach. While automated filtering enables processing larger datasets, the superior results from manually filtered data indicate that human oversight is crucial for achieving the next level of voice quality. This implies accepting reduced scalability

in favor of higher quality, as human verification of training data appears to be irreplaceable for optimal results.

Part 2: Emotion Recognition Results

Specific Text Version

In this version, the text content is specifically crafted to convey the intended emotion. For example:

Joy: "I just got promoted at work! This is the happiest day of my life!"
Sadness: "I had to say goodbye to my best friend today, I'm going to miss him."
Anger: "They completely destroyed the project I spent months working on!"
Surprise: "I can't believe I just won the lottery! This is unbelievable!"
Neutral: "The meeting is scheduled for three o'clock in the conference room."

Emotion Recognition Accuracy (Specific Text)

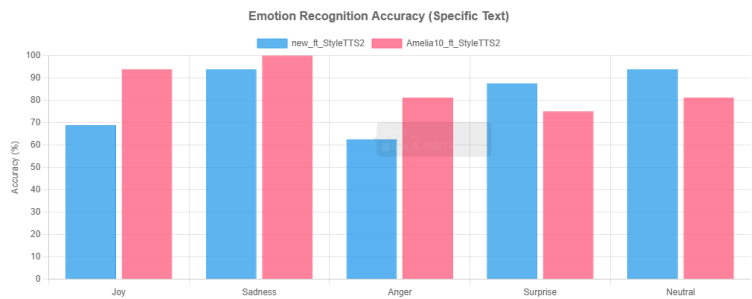
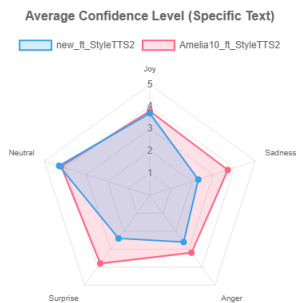


Figure 16 Emotion Recognition Results

Average Confidence When Correctly Identified (Specific Text)



Generic Text Version

In this version, the text content is neutral and doesn't inherently suggest any particular emotion. For example:

"This is a test sentence to compare different models and emotions."
"The quick brown fox jumps over the lazy dog."

This tests the model's ability to express emotions purely through voice characteristics, without relying on emotional content in the text.

Figure 17 Average Confidence When Correctly Identified

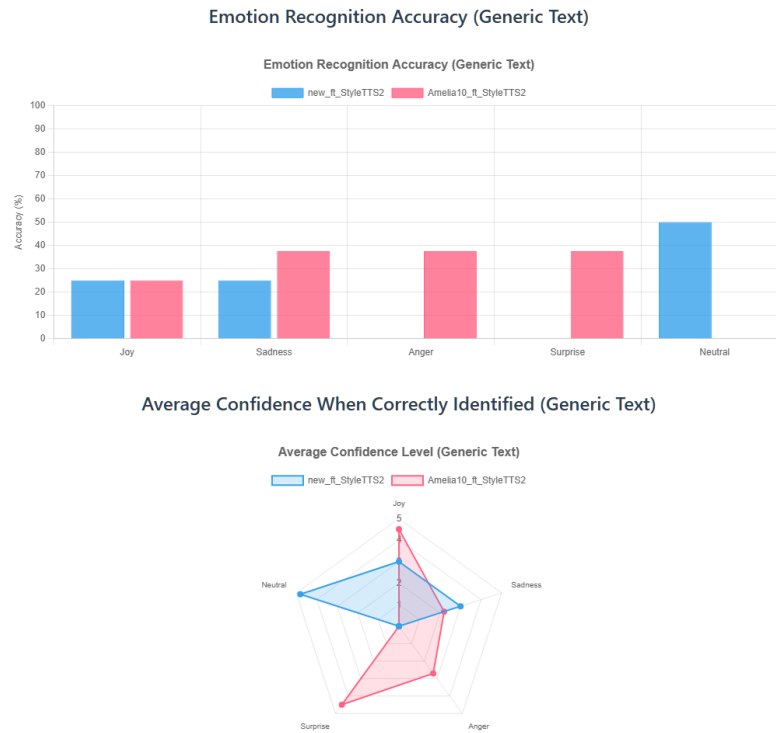


Figure 18 Recognition Accuracy

4. Animation Performance:

- Frame rate on recommended hardware
: 15 FPS
- Lip sync accuracy relative to phoneme detection

Different animation classes (happy, sad, surprise, angry) control mouth parameters for emotional expressions with different parameters .



Figure 19 Avatar face : Amelia Watson



Figure 20 Avatar face : Ike Eveland

- Quality of expression transitions

: The animation transition on the current implementation cause big no performance impact, but a spike of gpu memory usage can be expected because there must be a transition of removing the cache of the previous animations and start loading the new expression since having two expressions at the same time causes significant performance overload.

5.3 Detailed Performance Analysis

Further analysis of the system's performance has revealed important patterns and insights that guide ongoing development:

TTS Model Performance Factors:

1. Data Quality Impact:

Analysis of the relationship between training data quality and voice model performance suggests that:

- Carefully filtered datasets produce noticeably better voice quality
- Manual filtering provides significant improvements over automated filtering alone
- There appears to be a minimum viable dataset size for acceptable results
- Dataset diversity in terms of emotional range and speech contexts affects overall quality

2. Voice Characteristic Variations:

The system exhibits different levels of performance across voice types:

- Certain vocal characteristics appear easier to reproduce than others
- Distinctive speech patterns show differing levels of reproduction fidelity
- Emotional range reproduction varies between character models
- Unusual vocal techniques remain particularly challenging to replicate

Avatar Animation Analysis:

1. Expression Reproduction:

Facial expression reproduction effectiveness varies across expression types:

- - Some expressions (neutral, happy) are more consistently reproduced
- - More complex expressions (surprise, anger) show variable reproduction quality
- - Subtle expressions and transitions present ongoing challenges

2. Performance Bottlenecks:

System profiling has identified several key performance bottlenecks:

- Real-time rendering demands on GPU resources (12 GB of VRAM during transition)
- CPU requirements for animation parameter calculation
 - minimum requirements : Intel Core i5 (6 cores or more)
 - recommended requirements : Intel Core i7 / i9 (10 cores or more)
- Memory usage patterns during concurrent TTS and animation
 - Memory usage spikes when there is the animation transition

: mostly spiked on the gpu memory usage

- Initial loading times affected by storage performance
 - Initial loading time on low end hardware : 20 – 40 seconds
 - Initial loading time of high end hardware : 5-10 seconds

Conversation Quality Factors:

1. Response Relevance:

Analysis of conversation logs has revealed patterns in response quality:

- Varying degrees of direct relevance to user queries
- Character consistency challenges in extended conversations
- Context window limitations affecting coherence in longer interactions
- Variable accuracy in representing VTuber-specific knowledge

The containerized Ollama approach has demonstrated several benefits:

- Improved inference latency compared to native Windows execution
- Performance stability during sustained usage
- More efficient resource utilization on Windows systems
- Simplified deployment and configuration management

These findings continue to inform development priorities, highlighting both the current strengths of the implementation and the areas most in need of improvement.

Chapter 6: Conclusion

The Vbot project represents a significant advancement in AI-powered virtual entertainment, successfully integrating multiple complex technologies to create interactive VTuber experiences. Through careful implementation and optimization, the system achieves several key objectives:

Voice Synthesis Excellence: The fine-tuned StyleTTS2 models effectively reproduce VTuber voices with recognizable characteristics and emotional range, though quality varies based on training data availability and voice complexity.

Real-time Animation: THA4 implementation delivers responsive avatar animation with consistent 15 FPS performance and effective lip synchronization, operating within practical\op[kmn]hardware constraints for end-users.

Natural Interaction: Containerized Ollama deployment enables fluid conversation flow with sub-second response times, maintaining character consistency while managing computational resources efficiently.

However, several challenges remain:

1. **Data Quality Dependencies:** Manual intervention in training data preparation continues to limit scalability, particularly for voice model training.
2. **Resource Requirements:** Hardware demands, especially for GPU memory (minimum 8GB VRAM) and processing power, create accessibility barriers.
3. **Technical Limitations:** Current animation capabilities lack support for complex gestures and full-body movements common in VTuber performances.

Looking forward, the project establishes a foundation for future developments in several key areas:

1. **Enhanced Data Processing:** Development of improved automated filtering techniques to reduce manual intervention requirements.
2. **Extended Animation:** Implementation of more sophisticated movement and expression capabilities.
3. **Optimized Resource Usage:** Further refinement of containerization and memory management strategies.
4. **Ethical Framework:** Development of guidelines for responsible deployment and use of AI VTuber technology.

Vbot

The Vbot project demonstrates both the viability and current boundaries of AI-powered virtual character technology. As these technologies continue to evolve, careful consideration must be given to balancing technical advancement with ethical implications, ensuring that future developments enhance rather than diminish the unique value of human-performed virtual entertainment.

References

- Avatar model and voice : Ike Eveland, Nijisanji Corp.
- Avatar model: and voice: Amelia Watson, Hololive Corp,
- Khungurn, P. (2023). Talking Head Anime from a Single Image 4: Improved Models and Its Distillation. GitHub repository. <https://github.com/pkhungurn/talking-head-anime-4-demo>
- Li, Y., et al. (2022). StyleTTS 2: Towards Human-Level Text-to-Speech through Style Diffusion and Adversarial Training with Large Speech Language Models. GitHub repository. <https://github.com/yl4579/StyleTTS2>
- Baevski, A., Zhou, Y., Mohamed, A., & Auli, M. (2020). wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. *Advances in Neural Information Processing Systems*, 33.
- Qian, K., Zhang, Y., Chang, S., Yang, X., & Hasegawa-Johnson, M. (2019). AutoVC: Zero-Shot Voice Style Transfer with Only Autoencoder Loss. *International Conference on Machine Learning*.
- Chen, J., Tan, X., Luan, J., Qin, T., & Liu, T.-Y. (2021). HiFiSinger: Towards High-Fidelity Neural Singing Voice Synthesis. *arXiv preprint arXiv:2109.08996*.
- Kim, Y., et al. (2021). Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech. *International Conference on Machine Learning*.
- Isola, P., Zhu, J. Y., Zhou, T., & Efros, A. A. (2017). Image-to-Image Translation with Conditional Adversarial Networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.
- Hsu, W. N., et al. (2021). HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*.
- Brown, T. B., et al. (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 33.
- Brock, A., Donahue, J., & Simonyan, K. (2019). Large Scale GAN Training for High Fidelity Natural Image Synthesis. *International Conference on Learning Representations*.
- Jaegle, A., et al. (2021). Perceiver: General Perception with Iterative Attention. *International Conference on Machine Learning*.
- Ollama Project. (2023). Ollama: Get up and running with large language models locally. GitHub repository. <https://github.com/ollama/ollama>

Appendix A: User Manual for Vbot

A.1 System Requirements

A.1.1 Hardware Requirements

- Processor: Intel i5/AMD Ryzen 5 or better
- Memory: 16GB RAM minimum recommended
- Graphics Card: NVIDIA RTX series with at least 8GB VRAM
- Storage: 5GB of free disk space
- Internet Connection: Required for model downloads and Ollama API

A.1.2 Software Dependencies

- Operating System: Windows 10 (64-bit)
- Python: Version 3.10
- Docker Desktop: Required for running Ollama LLM service
- WSL 2: Windows Subsystem for Linux version 2
- CUDA Toolkit: CUDA 12.1 for GPU acceleration
- NVIDIA GPU Drivers: Latest compatible version

A.2 Installation Guide

A.2.1 Setting Up Python Environment

1. Install Python 3.10 from python.org
2. Clone the Vbot repository or extract the project files
3. Create a virtual environment:

```
python -m venv venv  
venv\Scripts\activate
```

4. Install the required dependencies:

```
pip install -r requirements.txt
```

A.2.2 Installing PyTorch with CUDA

1. Install PyTorch with CUDA support using the following command:

```
pip install torch==2.5.1 torchvision==0.20.1 torchaudio==2.5.1 --index-url  
https://download.pytorch.org/whl/cu121
```

2. Verify the installation by running:

```
python -c "import torch; print(torch.cuda.is_available())"
```

This should output **True** if CUDA is properly configured.

A.2.3 Configuring Docker

Install NVIDIA GPU Driver

1. Visit NVIDIA Driver Downloads
2. Select your GPU model and Windows 10
3. Download and install the driver
4. Verify installation by running `nvidia-smi` in Command Prompt

Install Docker Desktop

1. Download Docker Desktop
2. During installation:
 - Check "Use WSL 2 instead of Hyper-V"
 - Complete the installation
 - Restart your computer if prompted

Install WSL 2

1. Open PowerShell as Administrator
2. Run:

```
wsl --install -d Ubuntu
```

3. When prompted:
 - Create a username (use lowercase letters only, e.g., **user**)
 - Set a password (characters won't be visible as you type)
 - Wait for Ubuntu installation to complete

Configure Docker Desktop

1. Open Docker Desktop
2. Go to Settings (gear icon)
3. Under "Resources" → "WSL Integration":
 - Enable "Use the WSL 2 based engine"
 - Enable "Ubuntu" in the distribution list

Click "Apply"

4. Under "Docker Engine":
 - Click the edit button (it's a JSON file)
 - Replace ALL content with this exact configuration:

```
{
  "builder": {
    "gc": {
      "defaultKeepStorage": "20GB",
      "enabled": true
    }
  }
}
```

```

    },
    "experimental": false,
    "default-runtime": "nvidia",
    "runtimes": {
      "nvidia": {
        "path": "nvidia-container-runtime",
        "runtimeArgs": []
      }
    }
  }
}

```

- Click "Apply & Restart"

- Wait for Docker Desktop to restart

A.3 Using the Application

A.3.1 Starting the Application

1. Ensure Docker Desktop is running
2. Activate the Python virtual environment:

```
venv\Scripts\activate
```

3. Launch the application:

```
python Vbot.py
```

4. Note: The first run will take longer as it automatically:
 - Sets up the Ollama Docker container
 - Downloads the Stheno model (~5GB)
 - Configures GPU acceleration
 - Creates necessary directories

A.3.2 Interface Overview

- Main Window: Displays the animated character avatar
- Subtitle Area: Shows the AI's responses as text below the avatar
- Text Input Bar: Located at the bottom of the window for typing messages
- Control Buttons:
 - Send (➤): Submits the text input
 - Voice Input (🎤): Toggles voice recording
 - Chat History (💡): Opens a window showing previous conversations
 - Switch Character (🔄): Changes between available character models (Amelia and Eveland)

A.3.3 Voice Input Features

1. Click the microphone button () to start recording
2. Speak clearly into your microphone
3. The recording will automatically stop after a brief silence
4. The system will process your speech and generate a response

A.3.4 Character Controls

- Character Switching: Click the switch button () to change between different character models
 - Amelia Watson: Detective with a yellow background theme
 - Ike Eveland: Novelist with a blue background theme
- Character Animation: Each character has unique idle animations and emotional expressions
- Lip Synchronization: Character's mouth movements synchronize with the generated speech

A.4 Troubleshooting

A.4.1 Common Issues

- Application Fails to Start: Ensure Docker Desktop is running properly
- No Audio Output: Check your system's audio settings and ensure speakers are connected
- Slow Response Times: GPU acceleration may not be properly configured; verify CUDA installation
- Voice Input Not Working: Check microphone permissions and hardware connections
- Character Not Displaying: Ensure wxPython is properly installed and your GPU drivers are updated

A.4.2 Docker-Related Issues

1. Docker Desktop WSL Error
 - Open PowerShell as Administrator
 - Run: `wsl --update`
 - Restart Docker Desktop
2. NVIDIA Driver Issues
 - Verify GPU is detected: Run `nvidia-smi` in Command Prompt

- Update to latest driver if needed
3. Docker Container Not Starting
 - Check Docker Desktop is running
 - Try restarting Docker Desktop
 - Ensure WSL 2 is properly installed

A.4.3 Other Error Messages

- "Error: Model directory not found": Verify that the character model directories exist in the asset/model folder
- "Connection error - check if Ollama service is running": Ensure Docker Desktop is running
- "espeak error": If you encounter issues with phoneme generation, follow the espeak installation guide at <https://www.youtube.com/watch?v=BBlivx6o0WM>
<https://github.com/espeak-ng/espeak-ng/releases>